INTELIGÊNCIA ARTIFICIAL APLICADA

Instituto NTA

FASE 1 · ORQUESTRAÇÃO COM ESTADO

Aula 1.2

Workflows I: prompt chaining e routing

Os dois padrões mais simples, e a pergunta que organiza a fase inteira: quem controla o caminho, você ou o modelo?

Série: Engenharia de IA Aplicada · Agentes e RAG em Produção
Projeto: Núcleo de Agentes Operacionais
Duração de estudo: cerca de 1h30 · 25 min de teoria e 55 de código
Pré-requisito: Aula 1.1 (tag **aula-1.1**)

Prof. Thiago Azeredo Rodrigues · Série pública · 2026

1. Objetivo de aprendizagem

Ao final desta aula você terá implementado prompt chaining e routing primeiro em Python puro e depois como grafo, e saberá decidir, diante de um problema concreto, se ele pede um workflow ou um agente.

2. Por que esta aula importa

A Fase 0 inteira construiu um agente: o modelo decide qual ferramenta chamar, e você descobre o caminho depois, olhando o trace. É poderoso, e é o que a maior parte do mercado chama de IA aplicada hoje.

Também é, na maioria dos casos, exagero.

Quando um lead escreve relatando um erro de login, você já sabe o que precisa fazer: identificar quem é, consultar o cadastro, responder. O caminho é conhecido. Dar autonomia ao modelo para descobrir esse caminho a cada mensagem significa pagar mais caro, esperar mais tempo e aceitar variação onde você não queria variação nenhuma.

> **A PERGUNTA QUE ORGANIZA A FASE 1**
>
> **Quem controla o caminho?**
> **Workflow:** você controla. O caminho está escrito no código, e o modelo só preenche as etapas.
> **Agente:** o modelo controla. Ele escolhe a próxima ferramenta, e você não sabe de antemão por onde ele vai passar.

A regra que decorre disso vale o curso inteiro: **comece pelo padrão mais simples que resolve o problema, e só suba para autonomia quando ele exigir**. Boa parte do que se chama de agente por aí deveria ser um dos dois padrões desta aula.

3. Teoria

3.1 Prompt chaining

Decompor a tarefa em passos sequenciais e fixos: a saída de um alimenta o próximo. Use quando as etapas são conhecidas e verificáveis.

O ganho óbvio é custo. O ganho menos óbvio, e mais importante, é **qualidade**: cada passo recebe um contexto menor e uma instrução mais estreita, e por isso erra menos. Um prompt que tenta extrair, consultar e redigir de uma vez faz as três coisas pior do que três prompts que fazem uma cada.

Decompor não é só economizar. É uma técnica de precisão.

3.2 O gate, que é o que separa chaining de uma fila de chamadas

Entre dois passos cabe uma verificação: o resultado parcial serve para seguir adiante?

Sem gate, você leva dado ruim até o fim e descobre no último passo, depois de ter pago por todos. Com gate, você para onde o problema apareceu. No nosso fluxo, se a mensagem não trouxe o email, não adianta consultar o CRM nem redigir resposta: pergunta-se o email e encerra.

E um detalhe de projeto: **o gate é determinístico**. Verificar se uma string tem arroba é trabalho de código, não de modelo. Toda vez que você puder decidir com um `if`, decida com um `if`.

3.3 Routing

Classificar a entrada e mandar para o handler certo. Use quando os tipos de pedido são bem distintos e cada um tem tratamento próprio.

O ganho aqui também é de qualidade antes de ser de custo. Sem roteamento, você escreve um prompt gigante que tenta cobrir vendas, suporte e dúvidas ao mesmo tempo, com instruções que competem entre si. Com roteamento, cada handler fica com uma instrução curta e especializada.

> **ANALOGIA**
> Routing é a recepção de um escritório. Ela não resolve o seu problema: ela identifica qual é e te manda para a sala certa. Quem tenta fazer a recepção resolver tudo acaba com uma recepção ruim e salas vazias.

3.4 Como escolher, na prática

Padrão	Use quando	Custo típico
Chaining	As etapas são fixas e você as conhece de antemão	1 chamada por passo
Routing	Os tipos de pedido são distintos e têm tratamento próprio	1 chamada mais o handler
Agente	A tarefa é aberta e o número de passos não é conhecido	1 chamada por volta, com o histórico crescendo

Os três se combinam. Um router pode mandar o caso previsível para um chaining e o caso aberto para o agente da Fase 0. Essa combinação é o que a Fase 2 vai formalizar como supervisor.

3.5 E quando o grafo se paga

Esta aula implementa os dois padrões duas vezes, em Python puro e no LangGraph, e isso levanta uma pergunta honesta: se a versão pura funciona, por que desenhar um grafo?

Três respostas, e a terceira é a que importa. A topologia vira objeto e pode ser impressa e discutida. O estado vira explícito e pode ser persistido. E, principalmente: **um fluxo de três passos que nunca vai pausar não precisa de grafo. Um fluxo que precisa parar no meio para um humano aprovar, precisa.**

Quem desenha grafo para um chaining simples está pagando complexidade sem receber nada em troca. Saber disso é conteúdo da aula tanto quanto o código.

4. Mão na massa

Dois arquivos novos: `nucleo/fluxos.py` (Python puro) e `nucleo/grafo_fluxos.py` (os mesmos padrões como grafo). Nada da Fase 0 é alterado, e o `grafo.py` da Aula 1.1 também não.

Passo 1 — O contrato da triagem

O que e por quê: os dois padrões precisam transformar texto livre em dado estruturado. O contrato é o mesmo, e a classificação do router já cabe dentro dele.

```
class Triagem(BaseModel):
    """O que extraimos da mensagem do lead ANTES de decidir o que fazer."""

    intencao: Literal["comercial", "suporte", "duvida"] = Field(
        description="comercial: quer comprar, migrar de plano ou falar "
                    "com vendas. suporte: relatou erro, falha ou "
                    "indisponibilidade. duvida: pergunta sobre o "
                    "produto que nao e problema tecnico")

    email: str = Field(
        description="Email do lead, se aparecer na mensagem. "
                    "String vazia se nao aparecer")

    urgencia: Literal["baixa", "media", "alta"] = Field(
        description="Urgencia percebida na mensagem")

    resumo: str = Field(description="O pedido em uma frase")
```

O que faz: reaproveita o `extrair_estruturado` da Aula 0.6 inteiro, com o ciclo de correção quando o modelo erra o formato. Repare nas descrições de cada campo: elas são prompt, pela regra da Aula 0.5, e é delas que sai a acurácia da classificação.

Uma decisão de arquivo: o contrato mora em `fluxos.py`, não em `modelos.py`. Contrato específico de fluxo mora com o fluxo; o `modelos.py` guarda o contrato de **saída** do Núcleo, que outros sistemas consomem. E a Fase 1 não mexe em arquivo da Fase 0.

Passo 2 — O gate e o passo determinístico

```
def _gate_tem_email(triagem):
    """O GATE entre os passos.

    Um chaining sem gate e so uma sequencia de chamadas. O gate e o que
    permite PARAR no meio quando o resultado parcial nao serve, em vez
    de levar lixo adiante e descobrir no fim.

    Este e deterministico de proposito: checar se uma string tem "@" e
    trabalho de codigo, nao de modelo.
    """
    return "@" in triagem.email
```

```
def _passo_consultar(triagem, consultar_crm):
    """PASSO 2 - enriquecer com dados do CRM.

    Repare na diferenca para a Fase 0: aqui NAO perguntamos ao modelo
    se ele quer consultar o CRM. Nos sabemos que queremos. O caminho e
    nosso; o modelo so preenche as etapas.
    """
    with observar("consultar_crm", entrada=triagem.email) as sp:
        dados = consultar_crm(email=triagem.email)
        if sp is not None:
            sp.saida = dados
        return dados
```

O que faz: leia o comentário do segundo bloco, porque ele é a aula inteira em três linhas. Na Fase 0 nós **perguntávamos ao modelo** se ele queria consultar o CRM. Aqui nós sabemos que queremos. O caminho é nosso; o modelo só preenche as etapas.

Passo 3 — O chaining montado

```
def triar_mensagem(mensagem, consultar_crm, provider="anthropic"):
    """O chaining de tres passos, com um gate entre o primeiro e o segundo.

    extrair -> [gate] -> consultar CRM -> redigir

    Devolve (resposta, triagem). Quando o gate barra, a resposta diz o
    que falta, e os passos 2 e 3 nem chegam a rodar.
    """
    with iniciar_trace("chaining", entrada=mensagem):
        triagem = _passo_extrair(mensagem, provider)

        if not _gate_tem_email(triagem):
            # Parar cedo e barato. Seguir com dado faltando e caro.
            return ("Nao identifiquei o email na mensagem. Poderia "
                    "confirmar o email de cadastro?"), triagem

        dados = _passo_consultar(triagem, consultar_crm)
        resposta = _passo_redigir(triagem, dados, provider)
        return resposta, triagem
```

Resultado esperado: três passos no caso feliz, e um só quando o gate barra. O trace mostra isso de graça, porque a observabilidade da Aula 0.7 atravessou intacta:

```
[TRACE] chaining  1ms
  [span] extrair  0ms
  [span] consultar_crm  0ms
  [gen] redigir  0ms

resumo: {'chamadas_ao_modelo': 1, 'ferramentas_executadas': 2, ...}
```

Repare no resumo: **uma** chamada ao modelo no caminho inteiro, contra as quatro que o agente da Fase 0 gastava no mesmo caso. A extração conta como ferramenta aqui porque é um passo nosso, não uma decisão dele.

Passo 4 — O router

```
def rotear(mensagem, handlers, provider="anthropic"):
    """Classifica a mensagem e entrega ao handler da intencao.

    handlers -> dict {"comercial": fn, "suporte": fn, "duvida": fn}

    Cada handler recebe a Triagem ja pronta e devolve a resposta. O
    roteador nao sabe o que cada um faz, e e assim que deve ser: trocar
    um handler nao mexe no roteador.
    """
    with iniciar_trace("routing", entrada=mensagem) as raiz:
        triagem = classificar(mensagem, provider)

        handler = handlers.get(triagem.intencao)
        if handler is None:
            # Rota sem handler e erro de configuracao, nao de runtime:
            # falha alto, como na Aula 0.1.
            raise ValueError(
                f"Sem handler para a intencao '{triagem.intencao}'. "
                f"Handlers registrados: {sorted(handlers)}")

        with observar(f"handler:{triagem.intencao}") as sp:
            resposta = handler(triagem)
```

```
            if sp is not None:
                sp.saida = resposta

        if raiz is not None:
            raiz.saida = resposta
        return resposta, triagem
```

O que faz: o roteador não sabe o que cada handler faz, e é assim que deve ser: trocar um handler não mexe no roteador. E rota sem handler **levanta exceção**, porque é erro de configuração, não condição de runtime. É a regra de falhar alto da Aula 0.1.

Passo 5 — Os mesmos dois, como grafo

O que e por quê: no grafo, o `if` do gate vira uma aresta condicional, e o dict de handlers vira topologia.

```
    def gate(estado: EstadoChaining) -> str:
        triagem = Triagem(**estado["triagem"])
        return "consultar" if _gate_tem_email(triagem) else "pedir_email"

    g = StateGraph(EstadoChaining)
    g.add_node("extrair", extrair)
    g.add_node("consultar", consultar)
    g.add_node("redigir", redigir)
    g.add_node("pedir_email", pedir_email)

    g.add_edge(START, "extrair")
    g.add_conditional_edges("extrair", gate,
                            {"consultar": "consultar",
                             "pedir_email": "pedir_email"})
    g.add_edge("consultar", "redigir")   # caminho FIXO: a marca do chaining
    g.add_edge("redigir", END)
    g.add_edge("pedir_email", END)

    return g.compile(checkpointer=checkpointer)
```

E o chaining desenhado, impresso pelo próprio código:

```
graph TD;
    __start__ --> extrair;
    extrair -.-> consultar;      (gate: tem email)
    extrair -.-> pedir_email;    (gate: nao tem)
    consultar --> redigir;       (caminho FIXO)
    redigir --> __end__;
    pedir_email --> __end__;
```

Agora o router, onde o grafo mostra serviço de verdade: uma aresta condicional com três saídas.

```
    g = StateGraph(EstadoRouting)
    g.add_node("classificar", classificar_no)
    for nome, funcao in handlers.items():
        g.add_node(nome, _fazer_no(nome, funcao))
        g.add_edge(nome, END)

    g.add_edge(START, "classificar")
    # path_map com uma entrada por rota. O LangGraph valida que todo
    # destino existe: rota sem no vira erro na MONTAGEM, nao em
    # producao. E uma garantia que o dict de handlers nao dava.
    g.add_conditional_edges("classificar", lambda e: e["rota"],
                            {n: n for n in handlers})
```

```
    return g.compile(checkpointer=checkpointer)
```

```
graph TD;
    __start__ --> classificar;
    classificar -.-> comercial;
    classificar -.-> suporte;
    classificar -.-> duvida;
    comercial --> __end__;
    suporte --> __end__;
    duvida --> __end__;
```

O que faz: cada chave do dict de handlers vira um nó, então acrescentar uma rota é acrescentar uma entrada no dicionário. O grafo se monta sozinho a partir dele.

Passo 6 — Ver os quatro funcionando

```
# scratch/rascunho_1_2.py  (descartavel)
from nucleo import banco, ferramentas
from nucleo.fluxos import triar_mensagem, rotear
from nucleo.grafo_fluxos import rodar_chaining, rodar_router, construir_router

banco.criar_schema()

MENSAGEM = ("Oi, aqui e a Ana do TechFlow (ana@techflow.com). O login "
            "pelo SSO esta dando erro 500 desde ontem.")

# --- Padrao 1: chaining, nas duas versoes ---
r1, t1 = triar_mensagem(MENSAGEM, consultar_crm)
r2, t2, passos = rodar_chaining(MENSAGEM, consultar_crm)
print("puro :", r1)
print("grafo:", r2, "| passos:", passos)

# --- Padrao 2: routing ---
HANDLERS = {
    "comercial": lambda t: f"[vendas] proposta enviada para {t.email}",
    "suporte":   lambda t: f"[suporte] ticket {t.urgencia} aberto",
    "duvida":    lambda t: f"[faq] respondendo: {t.resumo}",
}
r3, t3 = rotear(MENSAGEM, HANDLERS)
print("rota :", t3.intencao, "->", r3)

# A topologia do router: tres saidas, desenhadas.
print(construir_router(HANDLERS).get_graph().draw_mermaid())
```

Passo 7 — Os testes que valem a aula

O primeiro é o mesmo da Aula 1.1, aplicado aqui: a versão pura e a versão em grafo precisam produzir o mesmo resultado.

```
# ---------- 3. O TESTE DA AULA: puro e grafo dao o mesmo resultado ----------

def test_chaining_puro_equivale_ao_grafo(monkeypatch):
    _modelo(monkeypatch)
    r_puro, t_puro = fluxos.triar_mensagem("A Ana reportou erro", _crm)

    _modelo(monkeypatch)
    r_grafo, t_grafo, passos = grafo_fluxos.rodar_chaining(
        "A Ana reportou erro", _crm)
```

```
    assert r_puro == r_grafo
    assert t_puro.model_dump() == t_grafo.model_dump()
    assert passos == ["extrair", "consultar", "redigir"]
```

E o segundo mede a tese da aula, em vez de afirmá-la:

```
    assert chamadas_fluxo["n"] == 2
    assert chamadas_agente["n"] == 3
    assert chamadas_fluxo["n"] < chamadas_agente["n"]
    # E a diferenca cresce com a tarefa: cada volta do agente reenvia o
    # historico inteiro (Aula 0.7), enquanto cada passo do workflow
    # recebe so o contexto daquele passo.
```

Resultado esperado: `pytest` mostra `77 passed, 2 deselected`.

E há um terceiro que merece atenção, porque revela um ganho concreto do grafo. Uma rota sem handler, na versão pura, só falha quando uma mensagem daquele tipo chega, o que em produção costuma ser num sábado. No grafo, o LangGraph valida na **compilação** que todo destino da aresta condicional existe como nó. Erro de configuração vira erro de inicialização.

Passo 8 — Commit e tag

```
pytest -v            # esperado: 77 passed, 2 deselected
git add .
git commit -m "Aula 1.2: workflows I, chaining e routing"
git tag aula-1.2
git push && git push --tags
```

5. Portabilidade

Chaining e routing não são conceitos de IA: são decomposição de tarefa e despacho por tipo, que você já faz em qualquer sistema. O que muda é que uma das etapas agora é uma chamada de modelo.

Por isso os dois atravessam qualquer stack. No LangChain eles aparecem como composição de runnables; em orquestradores de pipeline, como DAG com branch; num sistema de mensageria, como roteamento por tópico. O vocabulário muda, o desenho não.

E a regra de decisão é a mais portável de todas: **autonomia é custo**. Você paga em dinheiro, em latência e em previsibilidade. Só compre quando o problema exigir.

6. Terminei quando

O critério da aula: **uma entrada é classificada e segue o caminho certo, nas duas versões.**

- O chaining percorre os três passos e devolve a resposta.
- Uma mensagem sem email faz o gate barrar, e os passos seguintes não rodam.
- As três intenções caem em três handlers diferentes, igual no puro e no grafo.
- Você imprime a topologia do router e vê as três saídas.
- Você sabe dizer, para um problema concreto, se ele pede workflow ou agente.

7. Erros comuns e troubleshooting

- **A classificação erra muito.** As descrições dos campos do contrato estão vagas. Elas são prompt (Aula 0.5), e no routing elas são *o* prompt: é ali que você diz o que distingue uma intenção da outra.
- **O gate deixa passar dado ruim.** Gate feito com chamada ao modelo quando um `if` resolveria. Verificação determinística se faz em código.
- **Rota sem handler estourando em produção.** Na versão pura é erro de runtime; no grafo, de compilação. Se você está na versão pura, teste todas as rotas.
- **O workflow ficou mais caro que o agente.** Normalmente é um passo que não precisava de modelo, ou um passo que reenvia contexto desnecessário. Olhe o trace e veja quantas generations existem.
- **Desenhou grafo e não ganhou nada.** Pode ser o diagnóstico certo. Se o fluxo não precisa pausar, retomar ou ser inspecionado, a versão pura é a resposta, e reconhecer isso é maturidade de engenharia.

8. Ponte para a próxima aula

Chaining e routing têm o caminho escrito de antemão. Mas e quando o número de subtarefas não é conhecido antes? Uma mensagem que traz três problemas diferentes não cabe num caminho fixo nem numa rota só.

Na **Aula 1.3, Workflows II**, entram paralelização, para rodar subtarefas independentes ao mesmo tempo, e orchestrator-workers, onde um orquestrador decompõe a tarefa dinamicamente, delega e sintetiza. É o último degrau antes da autonomia de verdade.